SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section introduces SmartFlow Orders, an agentic procurement solution designed to bridge the gap between complex user intent and actionable product ecosystems. While traditional e-commerce platforms focus on individual item searches, SmartFlow Orders addresses the "Compatibility Gap" identified in our Product Review Documentation by automating the research, validation, and bundling process through high-level AI reasoning.

1.1. Purpose:

*This document outlines the high-level architecture and technical design of **SmartFlow Orders**. It serves as a guide for the development team to ensure the "Agentic Procurement" logic is technically sound, scalable, and budget-efficient.*

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *A lightweight, client-side web application leveraging an "Agent-in-the-Loop" model.*
2. ***Data Flows:***
    a. *User intent flows from the HTML interface, is processed by JS, validated against a local JSON database, and optimized via the GLM API.*
3. ***Model Process:***
    a. *End-to-end workflow from intent parsing to "Smart Swapping" and final JSON export.*

1.2. Background:

Currently, users building complex bundles (like PC parts or smart homes) must manually verify compatibility across multiple tabs. SmartFlow Orders replaces this manual "Search & Check" process with an autonomous reasoning engine.

1. ***New Capabilities:** Real-time budget-constrained substitution and technical compatibility validation using LLM reasoning.*

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
End-User (Buyer)	*Enters project goals via natural language and reviews the AI-generated bundle.*	*Intuitive interface, accurate budget adherence, and zero manual compatibility checking.*

© UMHackathon 2026

E-comm Support Agent	*Uses the tool to resolve customer tickets by finding "Smart Swaps" for unavailable items.*	*High-speed alternative suggestions and verified stock status for replacements.*
Development Team	*Build and optimize the JS logic and Z AI (GLM) prompt templates.*	*Modular JavaScript code, clear separation of UI and AI logic, and efficient API handling.*
QA Team	*Validating the agent's reasoning, budget accuracy, and JSON export integrity.*	*Clear prompt/response test pairs, documented edge cases for "Smart Swaps," and schema validation.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web-Based Application (SPA)*
Architecture	*Client-Server (Agentic Interface)*

SmartFlow Orders is primarily structured as a **Client-Side Agentic System** optimized for rapid procurement. The frontend is built with **HTML5 and CSS3**, while the core logic is handled by a **Vanilla JavaScript Engine**. This engine acts as the orchestrator between the **Z AI (GLM) API** and a local **Mock Inventory Database (JSON)**.

Unlike traditional search-based stores, this architecture uses the LLM as a "Reasoning Layer" that dynamically filters inventory based on multi-dimensional constraints (Budget, Technical Specs, and Style). The system is designed to be lightweight and portable, capable of being deployed as a static site while maintaining complex "Stateful" memory of the user's shopping cart during the conversational refinement phase.

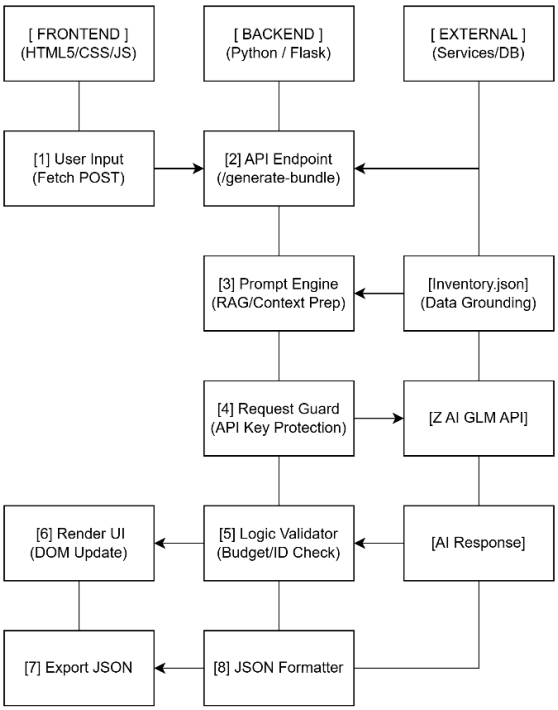
2.1.2 LLM as Service Layer

Z AI (GLM) is not just a chatbot; it acts as our **Reasoning Engine**. It sits between the user input and our local product logic, making decisions on which items to swap based on the constraints we provide in the system prompt.

2.1.3 Dependency Diagram

a. ***Prompt Construction:*** *JS captures user input and wraps it in a "System Instruction" containing the current* ***Inventory Schema****.*
b. ***Context Window:*** *Includes the last 3 user turns + the current cart_state (JSON).*
c. ***Parsing:*** *The system receives a structured string from GLM, which JS parses into a visual UI grid.*
d. ***Limits:*** *JS enforces a* ***2,000-character*** *limit on the frontend before the prompt is sent to avoid token overflow.*

© UMHackathon 2026



2.2.2. Sequence Diagram

Step	Component	Action	Technical Detail
1	*Frontend (JS)*	*Event Listener*	*JS listens for click on #send-btn or keypress (Enter) on #input-box.*
2	*Frontend (JS)*	*Payload Prep*	*JS grabs input.value, clears the box, and displays the user's text in the chat window.*
3	*Backend (Py)*	*Intent Extraction*	*Python receives the text and asks GLM: "Is this a new request, a swap, or a budget change?"*
4	*Backend (Py)*	*Data Fetch*	*Python queries products.json based on the AI's "Reasoning."*
5	*Frontend (JS)*	*Dynamic Injection*	*JS receives the JSON result and uses .innerHTML to inject product cards directly into the chat flow.*

2.2. Technological Stack

2.2.1. ***Frontend:*** *HTML, JS, CSS (React.js)*

2.2.2. ***Backend:*** *Python (Flask)*

2.2.3. ***Database:*** *Local (JSON)*

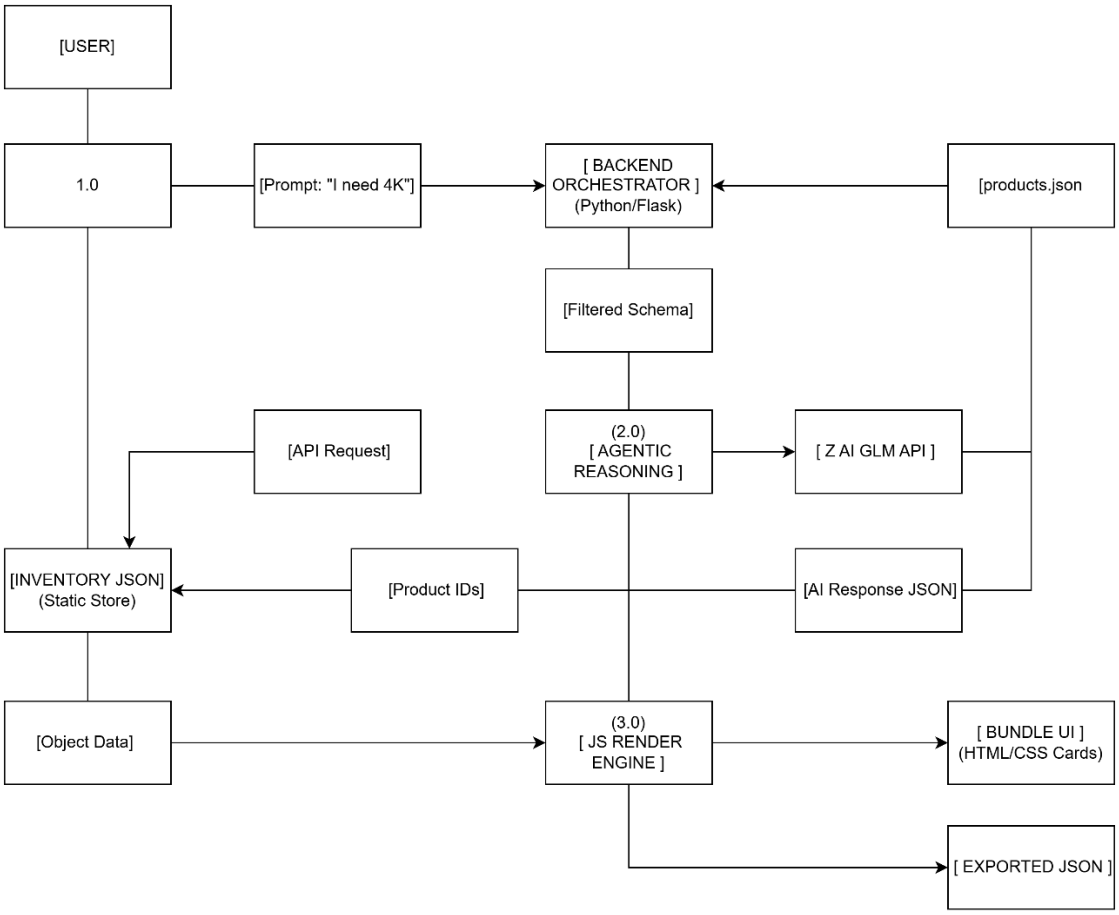
2.2.4. ***Deployment:*** *Vercel (frontend), Flask server (backend)*

© UMHackathon 2026

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

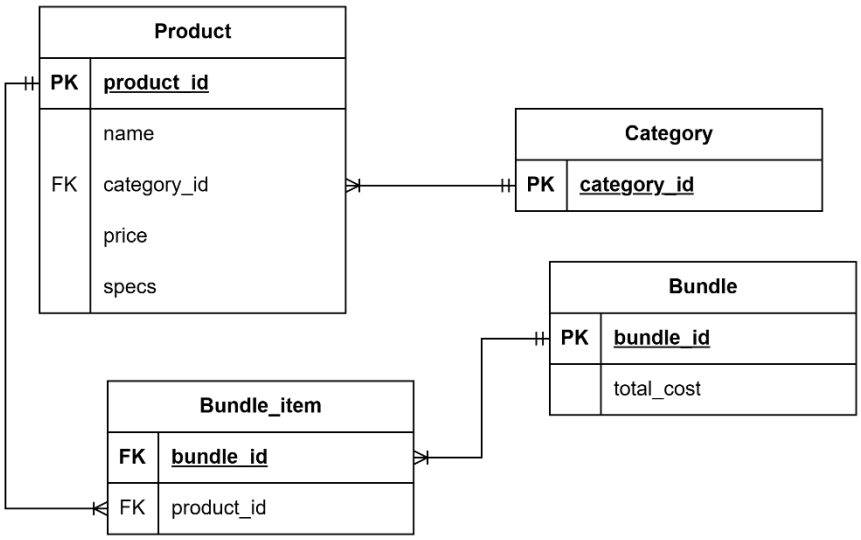
2.3.1.1. Data Flow Diagram (DFD)



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2. Normalized Database Schema



© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. 3 core features that our team is going to build and do a demo by the end of the UMHackathon 2026.

#	Features	Description
1	Intent Architect	Converts "I want to start a garden" into a list of 5 essential items.
2	Smart Swap	Automatically replaces an item if the user says "Too expensive."
3	Compatibility Check	Logic-based verification to ensure parts work together.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality	Requirements	Implementation
Security	API Credentials must never be exposed to the client.	Implementation: All Z AI calls are made server-side in Python; the frontend only sees the final result.
Performance	Initial bundle generation should take < 5 seconds.	Implementation: Python uses asynchronous processing for the AI call and local caching for the inventory database.
Reliability	The system must handle AI timeouts gracefully.	Implementation: Python try-except blocks return a "Server Busy" JSON to JS, which then displays a retry button in HTML.

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

- **Live Marketplace Integration**: Real-time API syncing with live vendors (e.g., Amazon, Shopify) to fetch dynamic pricing.

- **Collaborative Bundling**: Multi-user sessions where two people (e.g., a couple) can edit the same "Project Cart" simultaneously.

- **Visual AR Preview**: A feature to "see" the generated equipment bundle in a 3D space using Augmented Reality (AR) before purchasing.

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing****: For the deployment of the "Smart Swap" feature, we utilize a **Feature Flag** strategy. The agentic swapping logic is initially released to a small segment (10%) of users to monitor if the suggested alternatives meet user satisfaction. We simultaneously run **A/B Testing** on two different prompt strategies: "Strict Budget First" vs. "High Compatibility First" to determine which logic leads to a higher "Ready-for-Checkout" rate.*

4.1.2. ***Emergency Rollback (ER) & Golden Release****: In the event of an LLM service disruption or a surge in hallucinated responses (detected by our JS validation layer), an **Emergency Rollback** is triggered. The system automatically switches to a "Golden Release" version—a simplified, non-AI search interface that allows users to manually browse the inventory.json database, ensuring the platform remains functional even if the AI reasoning layer is offline.*

4.1.3. ***Priority Matrix:***

4.1.3.1. *Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project.*

- ***P1 (Critical):*** *AI suggests a product ID that does not exist in inventory.json.* ***Action:*** *Trigger auto-retry with "Strict Search" flag.*
- ***P2 (High):*** *Total cart cost exceeds user budget by > 5%.* ***Action:*** *Force a "Budget Correction" prompt to the agent.*

4.2. Monitoring:

Our monitoring strategy focuses on Health Checks and LLM Performance. Since the core of the system is the Python backend and Z AI integration, we monitor three specific layers:

4.2.1. *Real-Time Health & Damage Detection*

- ***Heartbeat Monitor:*** *A simple Python endpoint (/health) that the frontend pings every 120 seconds. If the backend fails to respond, the UI automatically triggers a **"Maintenance Mode"** overlay to prevent users from attempting a checkout.*
- ***API Latency Tracker:*** *We log the time taken for every Z AI GLM request.*
  - ***Trigger:*** *If the average response time exceeds **8 seconds** over a 5-minute window, the system automatically switches to a **"Compact Prompt Mode"** (reducing the amount of inventory data sent) to speed up inference.*
- ***Error Rate Threshold:*** *We monitor HTTP 500 (Internal Server Error) and 429 (Rate Limit) errors.*

© UMHackathon 2026

- ***Action:*** *If the error rate hits* ***>5%****, an automated alert is sent to the developer console, and the system temporarily falls back to a* ***Local Search Logic*** *(regex-based) instead of AI reasoning to ensure zero downtime.*

4.2.2. AI Integrity Monitoring

- ***Validation Logs:*** *Every "Smart Swap" is logged. If the JS validator detects that the AI is suggesting invalid IDs (hallucinations) in more than* ***3 consecutive turns****, the system clears the session context and restarts the agentic loop to flush out "context poisoning."*

4.3. Assumptions:

While developing SmartFlow Orders, we assume the following operational and environmental conditions are met:

1. ***Stable API Availability****: We assume the Z AI (GLM) API endpoint is functional and reachable from our deployment environment (e.g., Vercel/Heroku) with a standard API key.*
2. ***Valid Local Inventory****: We assume the products.json file is properly formatted and contains unique IDs. The system assumes that if a product is marked as stock: true, it is physically available.*
3. ***Client-Side Resource Capacity****: We assume the user's browser supports ES6+ JavaScript features (like async/await and Fetch API) and has sufficient memory to handle a JSON object of up to 500 items.*
4. ***Logical Intent****: We assume the user provides a prompt with at least one identifiable entity (e.g., "Camera," "Microphone," or "Budget"). The system assumes it does not need to handle completely nonsensical or "jailbreak" prompts in this version.*
5. ***Persistent Network Connection****: We assume the user has a stable internet connection during the "Reasoning" phase, as the agentic loop requires a consistent back-and-forth exchange between the JS frontend and Python backend.*

4.4. External Dependencies: *All the external tools that are applied to our system to function.*

Tools	*Purpose*	*Risk*	*Mitigation*
Z AI GLM API	*Core Reasoning Engine*	*High: Token cost/Rate limits*	*Caching common bundles in Python memory.*
Python (Flask/FastAPI)	*API Orchestration*	*Medium: Server crash*	*Automated restart scripts via deployment platform.*
Lucide/FontAwesome	*UI Icons*	*Low: CDN Failure*	*Bundling SVG icons locally in the CSS.*

© UMHackathon 2026

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

- ***Day 1-2**: SAD & QATD Finalization + UI Mockups.*
- ***Day 3-5**: Core JS Logic & JSON Database structure.*
- ***Day 6-8**: Z AI API Integration & Prompt Engineering.*
- ***Day 9-10**: Testing (QATD execution) & Presentation Prep.*

5.2. Team Members Role:

Name	Role	Core Responsibilities
Ng Xue Qi (Lead)	**AI Architect**	*Orchestrating the Python backend, designing "System Prompts" for GLM, and managing API security/token limits.*
Tan Xing	**Frontend Engineer**	*Crafting the HTML5/CSS3 interface, ensuring responsive layouts for the product grid, and implementing "Skeleton Loaders."*
Ang Qing Le	**Logic & Data Lead**	*Managing the JavaScript state (Cart history), developing the JSON search functions, and validating AI output against the database.*
Chong Ling	**Technical Writer/QA**	*Authoring the SAD and QATD, managing the Requirement Traceability Matrix, and coordinating end-to-end testing.*

© UMHackathon 2026